

Overfitting, and what to do about it

A model that memorises its training data can score perfectly and be useless. Today you watch it happen — training loss falling, validation loss climbing — and meet the five standard defences: less capacity, a weight penalty, dropout, stopping early, and the one that beats them all more often than you would expect.

Unit 4 · Real data

80 minutes

Lab 15 · overfit, then fix it

Compressible: cross-validation to reading

Seeing it

Five defences

Cross-validation

Lab 15

Before Tuesday

PART 1 · 15 MINUTES

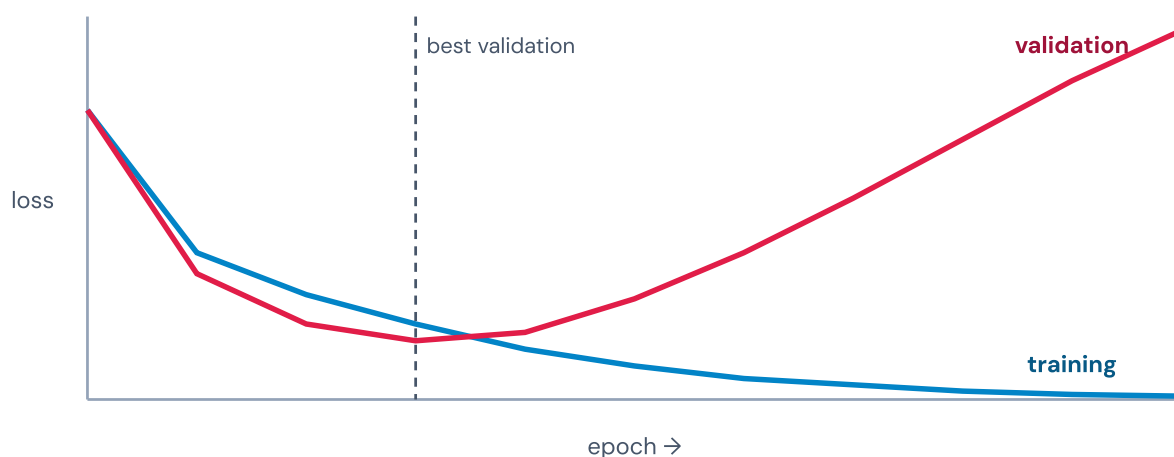
Training loss down, validation loss up

Seen, not described.

1

Two interleaved half-moons, noisy, 80 points to train on and 80 held back. A network with two hidden layers of 128 neurons — about 17,000 parameters for 80 points — trained for 400 epochs.

SKETCH OF LAB 15'S FIRST RUN — THE GAP AFTER THE DASHED LINE IS THE EVIDENCE



0.121

BEST VALIDATION LOSS
(EPOCH 38)

0.881

VALIDATION LOSS
AFTER 400 EPOCHS

96.3%

ACCURACY AT THE
BEST EPOCH

91.2%

ACCURACY AT THE
END

OVERFITTING, DEFINED BY ITS EVIDENCE

After the best epoch the network keeps improving on *these 80 points*, noise included, and gets worse on points it has not seen. **Memorising is not learning.** Training loss alone can never reveal it — only data the model was not fitted on can.

Three sets, three jobs

Set	Used for	Touched
Training	fitting the parameters	every step
Validation	choosing settings: epochs, penalty, size	as often as you like
Test	the one honest final number	once, at the end

Five defences

Each attacks the same gap from a different side.

2



1 · Less capacity

Fewer neurons or layers leave less room to memorise. One hidden layer of 4 neurons ends at a validation loss of 0.255.



2 · A weight penalty (L2)

Add $\alpha \cdot \sum w^2$ to the loss, so large weights must earn their size. $\alpha = 0.5$ ends at 0.146.



3 · Dropout

During training, switch off a random share of neurons each step, so no neuron can rely on a particular partner. One layer in PyTorch:

```
nn.Dropout(0.35)
```



4 · Early stopping

Keep the weights from the epoch with the best validation loss. Here: epoch 38, 96.3% instead of 91.2%.



5 · More data

The same big network on 1,000 training points ends at a validation loss of 0.124 — no cleverness at all.

Defence	Final validation loss	Validation accuracy
none	0.881	0.912
early stopping (epoch 38)	0.121	0.963
L2, $\alpha = 0.5$	0.146	0.963
smaller network (4 units)	0.255	0.912
1,000 training points	0.124	0.950

WHY MORE DATA BEATS MORE CLEVERNESS

Every other defence restrains the model so it cannot fit the noise. More data dilutes the noise so there is less of it to fit. When collecting more examples is possible, it is usually the most reliable improvement available — which is also why session 23 borrows what a network learned from over a million images.



Tuning on validation creates a new, smaller leak

Try enough settings against the same 80 validation points and you will find one that fits *them* by luck. That is why the test set stays untouched until the end — and why cross-validation exists.

PART 3 · 15 MINUTES · COMPRESSIBLE TO READING

Cross-validation

When the data is too small to spare a validation set.

3

Split the training data into k folds. Train on $k - 1$, validate on the one left out, and rotate until every fold has been the validation set once. Report the mean and the spread. On session 20's Titanic pipeline, 5-fold cross-validation gives:

```
cross_val_score(pipe, X, y, cv=5)
```

TEXT

```
fold accuracies: 0.777 0.787 0.781 0.770 0.826
mean 0.788 standard deviation 0.020
```

THE SPREAD IS THE POINT

One lucky split could have reported 0.826; another 0.770. The standard deviation tells you how much of any difference between two models could be noise. **A 1-point improvement with a 2-point spread is not an improvement yet.**

LAB 15 · 22 MINUTES IN CLASS

Overfit on purpose, then fix it

4

Lab 15 — `CSC4315_Lab15_Overfit_Then_Fix_It` . scikit-learn on a laptop CPU, under a minute. Overfit on purpose, then three fixes — a penalty, early stopping, less capacity — in one figure and one table.

Before Tuesday

1. **Finish Lab 15.**
2. **Read** Computer Vision series L1 (Images as Data) and L2 (Convolution) — Tuesday assumes them.
3. **Start Lab 16's first cell** on Colab before class: it downloads a dataset and a pretrained network.



Session 23 — Tuesday 24 November, 16:00

Images at realistic scale: why a dense network on pixels does not scale, what a convolutional filter does about it, and borrowing a network that has already learned to see.